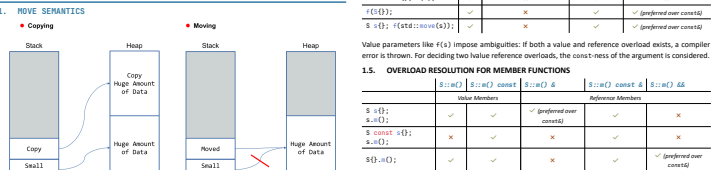
C++ Advanced | CPLA

Zusammenfassung

1. MOVE SEMANTICS



Sometimes, it is desirable to avoid copying values around for *performance reasons*.
– With a *copy*, the stack and heap data is copied. The new element is completely independent of the old one.
– With a *move*, only the stack data is copied. The old heap pointer gets deleted and the new one is attached to the same heap data. *Attention:* The old stack data is still valid, but in a *indeterminate* state. It now contains a *dangling pointer* that should not be used, otherwise *memory corruption is likely!*

Ownership Transfer: Resources of *expiring values* can be transferred to a new owner. Might be *more efficient* than a (deep) copy and destroying the original. Might be feasible when copying is not *(e.g. std::unique_ptr)*. Examples of such resources: Heap Memory, Locks, (File) Handles.

1.1. EXAMPLE MOVE CONSTRUCTOR (TESTAT 1)

```
struct ContainerForBigObject {
  ContainerForBigObject() // Default constructor
  : resource(std::make_unique<BigObject>()) {} // make_unique creates heap obj. & pointer

  ContainerForBigObject(ContainerForBigObject const& other) // Copy constructor
  : resource(std::make_unique<BigObject>(*other.resource)) {}
  // Creates a copy of the heap object and a new pointer to the copy

  ContainerForBigObject(ContainerForBigObject&& other) // Move constructor
  : resource(std::move(other.resource)) {}
  // New pointer points to the same heap object, 'other' is valid, but indeterminate state

  // Copy assignment operator
  auto operator=(ContainerForBigObject const& other) -> ContainerForBigObject& {
    resource = std::make_unique<BigObject>(*other.resource); // Same as copy constructor
    return *this; // The 'this' object gets returned
  }

  // Move assignment operator
  auto operator=(ContainerForBigObject&& other) -> ContainerForBigObject& {
    using std::swap; // Search swap in namespace scope, fallback to std-implementation
    swap(resource, other.resource); // Pointers get swapped
    //resource = std::move(other.resource) is possible too, same as move constructor
    return *this;
  }
private:
  std::unique_ptr<BigObject> resource;
};
```

1.2. LVALUE AND RVALUE REFERENCES

Lvalue: Everything that has an identity *(e.name)*. The address can be taken. **Lvalue reference usages:** Function parameter type *(no copy happens, but side-effects on argument are possible)*, Member/local variable or as a return type *(returned object must survive, e.g. a reference parameter or vector.at())*. Beware of dangling references!
Rvalue: Disposable values without an address or a name. Either a literal, a temporary object or an explicitly converted lvalue. If an rvalue is passed to a rvalue reference parameter, it gets a name and turns into an lvalue. See Chapter 2.2. "Perfect forwarding with std::forward".

lvalue references	*rvalue references*
Binds to an lvalue *(everything with a name)*	Binds to an rvalue *(temporary objects, literals)*
`T &`	`T &&`
The original must exist as long as it is referred to.	Can extend the life-time of a temporary.

`auto modify(T& t) -> void {`
`  // manipulate 't'`
`}`
`auto lvalueRefExample() -> void {`
`  T t = 5;`
`  modify(t); // modify(5) would not`
`  // work because 5 is not a reference`
`  T& lr = t; // rarely used`
`}`
	`auto createT() -> T;`
	`// 't' can be "stolen" (not needed anymore)`
	`auto consume(T&& t) -> void {`
	`  // manipulate 't'`
	`}`
	`auto rvalueRefExample() -> void {`
	`  consume(T{}); // new temporary T as param`
	`  T&& t = create(); // rarely used`
	`}`

lvalue examples	***rvalue examples***
The lvalue/rvalue is marked as [value]	
`// Has a name` `T value {}; std::cout << [value];`	`// Temporary object without a name` `int value{}; std::cout << [value + 1];`

`// Function calls returning a lvalue ref`
`[std::cout << 23]; // returns 'std::cout &'`
`[vec.front()]; // returns 'T &'`
	`// Function call returning a value type`
	`std::abs(int n); // returns 'int'`

`// Built-in prefix inc/dec expressions` `++a; // returns 'T &'`	`// Built-in postfix inc/dec expressions` `a++; // returns 'T', the value without +1`

`// lvalue ref, but has a name`
`auto foo(T& param) -> void {`
`  std::cout << [param]; }`
	`// Temporary T without a reference`
	`auto create() -> T; [create()];`

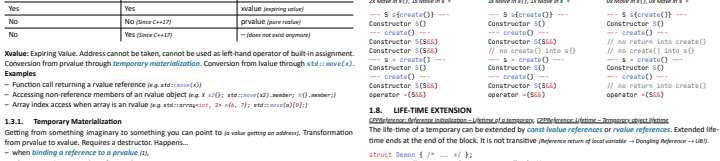
`// rvalue ref, but has a name`
`auto print(T& param) -> void {`
`  std::cout << [param]; }`
	`// Transformation into rvalue with move`
	`T value{}; T o = [std::move(value)];`

`// References have an address` `T& create(); [create()];`	`// xvalue, binds to rvalue references` `T&& create(); [create()];`
`// String literals are always lvalues` `std::cout << ["Hello"];`	**Rule of thumb:** Does element keep living? ✓ lvalue *(only copy)*, ✗ rvalue *(copy & move possible)*

`// depends on the implementation of a`
`T value{}; std::cout << [value + 1];`

1.3. VALUE CATEGORIES

CPPReference: Value categories



A value is always either a lvalue, xvalue or prvalue. Lvalue and rvalue are also known as "non-the left side of an assignment", because the left side can't always be assigned: `int const a = 0; a = 7; // error`

Has identity?	*Can be moved from?*	*Value category*
Yes	No	lvalue
Yes	Yes	xvalue *(expiring value)*
No	No *(Since C++17)*	prvalue *(pure value)*
No	Yes *(Since C++17)*	→ *(does not exist anymore)*

Xvalue: Expiring Value. Address cannot be taken, cannot be used as left-hand operator of built-in assignment. Conversion from prvalue through *temporary materialization*. Conversion from lvalue through `std::move(x)`.
Examples
– Function call returning a rvalue reference *(e.g. std::move(x))*
– Accessing non-reference members of an rvalue object *(e.g. s2[2]; std::move(x2).member; *f().member;)*
– Array index access when array is an rvalue *(e.g. std::array<int, 2> e{6, 7}; std::move(e)[0];)*

1.3.1. Temporary Materialization

Getting from something imaginary to something you can point to *(a value getting an address)*. Transformation from prvalue to xvalue. Requires a destructor. Happens...
– when *binding a reference to a prvalue (1)*,
– when *accessing a member of a prvalue (2)*,
– when accessing an element of a prvalue array *(int value = {int{}}[10, 20][1]; // value = 20)*,
– when converting a prvalue to a pointer *(const int &ref = (5-5); const int *ptr = &ref;)*
– when initializing an std::initializer_list<T> from a braced-init-list. *(std::vector<i_t,i_t>;)*

```
struct Ghost {
  auto haunt() const -> void { std::cout << "booo!\n"; } // ~Ghost() = delete; would error
};
auto evoke() -> Ghost { return Ghost{}; }
auto main() -> int { Ghost&& sam = evoke(); /* (1) */ Ghost{}.haunt(); /* (2) */ }
```

const lvalue reference: Binds lvalues, xvalues and prvalues: `auto f(T const &) -> void`

1.4. OVERLOAD RESOLUTION FOR FREE FUNCTIONS

	f(S)	*f(S &)*	*f(S const &)*	*f(S &&)*
`S s{}; f(s);`	✓	✓ *(preferred over const&)*	✓	×
`$ const s{}; f(s);`	✓	×	✓	×
`f(S{});`	✓	×	✓	✓ *(preferred over const&)*
`S s{}; f(std::move(s));`	✓	×	✓	✓ *(preferred over const&)*

Value parameters like `f(s)` impose ambiguities: If both a value and reference overload exists, a compiler error is thrown. For deciding two lvalue reference overloads, the const-ness of the argument is considered.

1.5. OVERLOAD RESOLUTION FOR MEMBER FUNCTIONS

	S::m()	*S::m() const*	*S::m() &*	*S::m() const &*	*S::m() &&*
	Value Members		*Reference Members*		
`S s{};` `s.m();`	✓	✓	✓ *(preferred over const&)*	✓	×
`$ const s{};` `s.m();`	×	✓	×	✓	×
`S{}.m();`	✓	✓	×	✓	✓ *(preferred over const&)*
`S s{};` `std::move(s).m();`	✓	✓	×	✓	✓ *(preferred over const&)*

Reference and non-reference overloads cannot be mixed. The reference qualifier affects the `this` object and the overload resolution. `S::m() const &&` is theoretically possible, but an artificial case.

1.6. SPECIAL MEMBER FUNCTIONS

CPPReference: Non-static member functions, *CPPReference: Special member functions*, *CPPReference: Constructors and member initializer lists*, *CPPReference: Assignment Operators*, *CPPReference: The rule of three/five/zero*

– **Constructors:** Default Constructor, Copy Constructor, Move Constructor *(Called on variable initialization)*
– **Assignment Operators (=, +=, ++ etc.):** Copy Assignment, Move Assignment *(Called on variable reassignment)*
– **Destructors (~)** *(Called automatically when variable goes out of scope to clean up the objects resources)*

It is normally not necessary to implement these yourself (*"rule of zero"*), but if a destructor or a copy function is needed, all three are needed (*"rule of three"*). For further optimization, the move functions should also be implemented (*"rule of five"*). Copy Constructor/Assignment should be marked const *(they don't modify this)*.

Assignment operators must be *member functions*. Move operations *must not throw exceptions*, thus aren't allowed to allocate memory. Use the default implementation of the special members whenever possible.

1.6.1. Move Constructor `$($ &&)`

CPPReference: Move constructor

Takes the guts out of the argument and moves them to the constructed object. Leaves the argument in *valid but indeterminate state*. Don't use the argument after it has been moved from until you assign it a new value.
Default behavior: Initialize base classes and members with move initialization: `:$($ && v) : m(std::move(v.m))`

1.6.2. Copy/Move assignment `auto operator=($ const &) -> $&` / `auto operator=($ &&) -> $&`

CPPReference: Copy assignment, *CPPReference: Move assignment*

Copies/Moves the argument into the `this` object. Executed when the variable to copy/move to has *already been initialized*. Must be a member operator. **Default behavior:** Initializes base classes and members with copy-assignment/move-assignment.

1.6.3. Destructor `~$()`

CPPReference: Destructors

Deallocates resources held by the `this` object. No parameters. Must not throw *(is noexcept)*. Must be implemented if a custom copy/move is defined. **Default behavior:** Calls destructor of base classes and members.

1.6.4. Copy-Swap-Idiom

CPPReference: Copy-swap

Allows efficient implementation of the copy-assignment operator. Utilizes the copy constructor to create a temporary object, and exchanges its contents with itself using a non-throwing swap. Therefore, it *swaps the old data with new data*. The temporary object is then destructed automatically. It needs a copy-constructor, a *move* constructor and a swap function to work. Should be marked `noexcept`.

```
struct S {
  // swap() member function - called by non-member swap() below
  auto swap($& other) noexcept -> void {
    using std::swap; // Fall back to std::swap if no user defined swap is available
    swap(member1, other.member1); // Calls std::swap or custom swap of member variable
  }
  // Copy assignment
  auto operator=($ const& s) -> $& {
    if (std::addressof(s) == this) { // Avoids unnecessary self-copy
      $ copy = s; // Can throw exception, this-object stays intact
      swap(copy);
    }
    return *this;
  }
  // Move assignment
  auto operator=($&& s) -> $& {
    if (std::addressof(s) == this) {
      consume(T{}); // new temporary T as param
      T&& t = create();
    }
    return *this;
  }
};
// Destructor is implicit

auto swap($& lhs, $& rhs) noexcept -> void { lhs.swap(rhs); } // Calls S::swap() above
```

1.6.5. What you write vs. what you get

*Watch out for the combinations in red. **default**ed are bugs, should be implemented yourself. You want to stay above the black line, avoid going below it! When using multiple constructor / assignment methods, deleted takes precedence over defaulted, undeclared over defaulted. Defining a constructor / assignment method as = default; makes the implicit default explicitly user decl.*

write/get	*default ctor*	*destructor*	*copy ctor*	*copy assign*	*move ctor*	*move assign*
nothing	defaulted	defaulted	defaulted	defaulted	defaulted	defaulted
any ctor	undeclared	defaulted	defaulted	defaulted	defaulted	defaulted
default ctor	user decl.	defaulted	defaulted	defaulted	defaulted	defaulted
destructor	defaulted	user decl.	defaulted!	defaulted!	undeclared	undeclared
copy ctor	undeclared	defaulted	user decl.	defaulted!	undeclared	undeclared
copy assign	defaulted	defaulted	defaulted!	user decl.	undeclared	undeclared
move ctor	undeclared	defaulted	deleted	deleted	user decl.	undeclared
move assign	defaulted	defaulted	deleted	deleted	undeclared	user decl.

1.7. COPY ELISION

CPPReference: copy elision

In some cases, the compiler is required to elide *(omit)* specific copy/move operations regardless of the side-effects of the corresponding special member functions (*"Mandatory elision"*). The omitted operations need not exist. This happens...
– In *initialization*, when the initializer is a prvalue: `$ s = ${5{}};` *(Only 1 constructor call, 0 copy operations)*
– When a function call *returns a prvalue*: *f5() gets initialized directly in new_se() instead of in create()*
`auto create() -> $ { return $4; }`
`auto main() -> int { $ new_se(create()); $ * sp = new $(create()); }`

1.7.1. Further optional elisions

– **NRVO:** *Named Return Value Optimization*. Return type is a value type, return expression is a local variable of the return type. The object is constructed in the location of the return value. The constructors *must still exist* - even if they are elided. `return std::move()` prevents NVRO.
– **throw Expression:** Return expression is a local variable from the innermost surrounding try block. The object is constructed in the location where it would be moved or copied to.
– **catch Clause:** If the caught type is the same as the object thrown, it accesses the object directly.

1.7.2. Example

```
auto create() -> $ { $ s{0}; std::cout << "\t --- create() ---\n"; return s; }
auto main() -> int {
  std::cout << "\t --- $ s{create()} ---\n"; $ s{create()};
  std::cout << "\t --- s = create() ---\n"; s = create();
}
```

Disabled elision *(C++14)*	**Only mandatory elision** *(C++17)*	**With constructor elision** *(C++17)*
2x Move in s(), 1x Move in s =	*1x Move in s(), 1x Move in s =*	*0x Move in s(), 0x Move in s =*

`--- $ s{create()} ---`
`Constructor $(S&&)`
`--- create() ---`
`Constructor $(S&&)`
`Constructor $(S&&)`
`--- s = create() ---`
`Constructor $()`
`--- create() ---`
`Constructor $(S&&)`
`operator =($&&)`
	`--- $ s{create()} ---`
	`Constructor $()`
	`--- create() ---`
	`Constructor $(S&&)`
	`Constructor $(S&&)`
	`// no create() into s{}`
	`--- s = create() ---`
	`Constructor $()`
	`--- create() ---`
	`Constructor $(S&&)`
	`operator =($&&)`
		`--- $ s{create()} ---`
		`Constructor $()`
		`--- create() ---`
		`// no return into create()`
		`// no create() into s{}`
		`--- s = create() ---`
		`Constructor $()`
		`--- create() ---`
		`// no return into create()`
		`operator =($&&)`

1.8. LIFE-TIME EXTENSION

CPPReference: Reference initialization – Lifetime of a temporary, *CPPReference: Lifetime – Temporary object lifetime*

The life-time of a temporary can be extended by *const lvalue references or rvalue references*. Extended lifetime ends at the end of the block. It is not transitive *(Reference return of local variable → Dangling Reference → UB!)*.

```
struct Demon { /* ... */ };
auto summon() -> Demon { return Demon{}; } // Creates a demon
auto countEyes(Demon const&) -> void { /* ... */ }

auto main() -> int {
  summon();          // Demon dies at the end of the statement
  countEyes(Demon{}); // Demon lives long enough for count_eyes to finish
  // life-time can be extended by const & -> flaughun lives until end of block
  Demon const&flaughun = summon();
  // life-time also be extended by && -> laznik lives until end of block
  Demon&& laznik = summon();
  } // Flaughun and laznik die here
```

2. TYPE DEDUCTION

2.1. FORWARDING REFERENCES AND TYPE DEDUCTION

CPPReference: Template argument deduction, *CPPReference: Reference declaration – Forwarding References*

In some contexts, `T&&` does not necessarily mean "rvalue reference". *Exceptions:* `auto &&` or `T&&` when template type deduction applies for type T. Here, lvalues can also bind to `T&&` as a `T&`. This binding only works for Method Templates, not Class Templates, and only for `T` `&&`, not `std::vector<T> &&` or `T const &&`.

```
template <typename T>                    // T and ParamType can be different types!
auto f(ParamType param) -> void;         // f(expr-);
```

Deduction of type T depends on the structure of the type of the corresponding parameter ParamType:

2.1.1. ParamType is a value type (T)

*(e.g. auto f(T param) → void) Note: Pointers (T *) are also value types*
1. `<expr>` is a reference type: ignore the reference
2. Ignore the rightmost `const` of `<expr>` *(char const * const -> char const *)*
3. Pattern match `<expr>`'s type against ParamType to figure out T

```
int x = 23; int const cx = x; int const & crx = x;

// calls   // instances                          // deduced Ts
f(x);      auto f(int param) -> void;            // T = int
f(cx);     auto f(int param) -> void;            // T = int
f(crx);    auto f(int param) -> void;            // T = int
f(&str);   auto f(char const * param) -> void;   // T = char const *
```

2.1.2. ParamType is a reference (T&)

(e.g. auto f(T & param) → void or auto f(T const & param) → void)
1. `<expr>` is a reference type: ignore the reference
2. Pattern match `<expr>`'s type against ParamType to figure out T

```
int x = 23; int const cx = x; int const & crx = x;

// calls   // instances                          // deduced Ts
f(x);      auto f(int & param) -> void;          // T = int
f(cx);     auto f(int const & param) -> void;    // T = int const
f(crx);    auto f(int const & param) -> void;    // T = int const
```

Examples for Const References: *auto f(T const & param) → void*

```
// calls   // instances                          // deduced Ts
f(x);      auto f(int const& param) -> void;     // T = int
f(cx);     auto f(int const& param) -> void;     // T = int
f(crx);    auto f(int const& param) -> void;     // T = int
```

2.1.3. ParamType is a forwarding reference (T&&)

(e.g. auto f(T && param) → void)
1. `<expr>` is an lvalue: T and ParamType become lvalue references *(Free 3 examples turn into T&)*
2. Otherwise *(if <expr> is an rvalue)*: Rules for references apply *(Fourth example)*

```
int x = 23; int const cx = x; int const & crx = x;

// calls   // instances                          // deduced Ts
f(x);      auto f(int & param) -> void;          // T = int &
f(cx);     auto f(int const & param) -> void;    // T = int const &
f(crx);    auto f(int const & param) -> void;    // T = int const &
f(27);     auto f(int && param) -> void;         // T = int
f("OST");  auto f(char const (&) [4] param) -> void;  // T = char const (&) [4]
```

2.1.4. Type Deduction and Initializer Lists

CPPReference: std::initializer_list

When an `Initializer_list` is used for type deduction, an error occurs: `f({23});`
For this to work, a separate template is needed:

```
template <typename T>
auto f(std::initializer_list<T> param) -> void;
f({23}); // T = int, ParamType = std::initializer_list<int>
```

2.1.5. Type Deduction for `auto`

CPPReference: Placeholder type specifiers, *auto-(Dec|In)ver(ts|ted)*

Same deduction as above. `auto` takes the place of T:

```
auto x = 23;                  // T  -> int          auto is a value type
auto const cx = x;            // T  -> int const    auto is a value type
auto& rx = x;                 // T& -> int const &  auto is a reference type

auto&& uref1 = x;             // T&&: x  -> int (lvalue)      -> uref1 = int&
auto&& uref2 = cx;            // T&&: cx -> int const (lvalue) -> uref2 = int const&
auto&& uref3 = 23;            // T&&: 23 -> int&& (rvalue)     -> uref3 = int&&

// Special cases              // std::initializer_list<int>
auto lst1{23};                // int, was std::initializer_list<int> before C++17
auto lst1{3,23};              // Error, requires one single argument
```

`auto` **Return Type Deduction:** `auto` can be used as return type and for parameter declarations. Body must be available to deduce the type *(not possible in header)*. Multiple `auto` parameters are considered different types.

2.1.6. Type Deduction for `decltype`

CPPReference: decltype specifier

Represents the declared type of a named expression. `decltype(auto)` allows deduction of (inline) function return types, but *does not strip references (&) or const like plain auto*. Can take an expression for specifying trailing return types, but this requires the body of the function; it can therefore not be declared in a header.

```
int                 x       = 23;
int&                rx      = x;
decltype(rx)        rx_too  = rx; // rx_too  -> int&
auto                just_x  = rx; // just_x  -> int
decltype(auto)      more_rx = rx; // more_rx -> int&
template <typename Container, typename Index>
auto access(Container &c, Index i) -> decltype(auto) { return c[i]; } // Requires body
auto access(Container &c, Index i) -> decltype(c[i]) { ... } // Body not required (.hpp)
```

2.1.7. Type Deduction in Lambdas

Slides: Page 38

2.2. PERFECT FORWARDING WITH STD::FORWARD

CPPReference: std::forward, *Slides Page 25 onward* `#include <utility>`

```
template <typename T>
decltype(auto) forward(std::remove_reference_t<T>& param) {
  return static_cast<T&&>(param);
}
```

`std::forward` is a *conditional cast to an rvalue reference*. This allows arguments to be treated as what they originally were: lvalues remain lvalues and rvalues remain rvalues.

– If T is of *rvalue type*, T && is an *rvalue reference* in the return expression *(int -> int&&)*.
– If T is of *lvalue reference type*, the resulting type is an *rvalue reference to an lvalue reference* *(T & -> T & && Becomes: T & int & &. This would return "int & &" which can be collapsed into "int &")*.

```
template<typename T>
static auto make_buffer(T && value) -> BoundedBuffer<value_type> {
  BoundedBuffer<value_type> new_buffer{};
  new_buffer.push(std::forward<T>(value), // Use forward to insert elements as they are
  return new_buffer; }
```

2.3. STD::MOVE

CPPReference: std::move, *Slides Page 43 onward* `#include <utility>`

```
template <typename T>
decltype(auto) move(T&& param) { // remove_reference_t gets rid of lvalue references
  return static_cast<std::remove_reference_t<T>&&>(param);
}
```

Does not actually move objects. It's just a *unconditional cast to an rvalue reference*. This allows resolution of rvalue reference overloads *(auto f(T&& t))* and move-constructor/-assignment operations. *Caution! Moving a const object or a non-const object without a move constructor/assignment results in a copy, not move operation!* To allow for moving, there should be no `const` member in a class.
Reference Collapsing: "T& &", "T& &&" and "T&& &" become "T&", "T&& &&" becomes "T&&".

3. HEAP MEMORY MANAGEMENT

Lifetime on Stack: Deterministic, local variables get deleted automatically upon leaving their scope `{ int a; }`
Lifetime on Heap: Creation and deletion happens *explicitly* with `new` and `delete` *(Dangerous, avoid!)*.
`auto foo() -> void { auto ip = new int[5]; /* ... */ delete ip;}`

Rules: Delete every object you allocated, do not delete an object twice or access a deleted object. UB!

3.1. EXPLICIT LIFE-TIME MANAGEMENT

Global and local variables have life-time implicitly managed by the program flow. Some resources can be allocated and deallocated explicitly. This is *error-prone*. **Guideline:** Always wrap explicit resource management in an object which has implicit life-time management (RAII).

3.2. POINTER SYNTAX

CPPReference: Pointer declaration

```
auto ip = new int[5];  // auto -> int *
int v = *ip;           // accessing value: v == 5
```

Heap Array Access:

```
auto arr = new int[5][6];  // 5 needs to be compile-time-constant, auto -> int *
int v = arr[4];            // accessing element (*arr + 4 pointer objects)
```

Direct Member Access via the `->` operator:

```
struct S {
  auto member() -> void { this->value = ...; } int value; // 'this': pointer to a instance
};
auto foo() -> void {
  $* sp = new $(); sp->member(); // equal to (*sp).member(); "." binds stronger than "*"
}
```

Pointer Parameters: Pointers can be used as parameters. Addresses can be taken with `&` *(Caution, operator could be overridden)* or `std::addressof()` *(Preferred, the rvalue overload is deleted to prevent taking their address)*.

```
auto foo(int* p) -> void {}
auto bar() -> void { int* ip = new int[5]; int local = 6; foo(&ip); foo(&local); }
```

More complicated examples with pointer to pointer to pointer to int: Slides page 25

Const Pointers: `const` Pointer can't be modified, but the object behind it may. `const` is on the right side of the `*`. The declaration is read from right to left.

```
int * const ipc;  // Can't change pointer address, but can change int
int const * icp;  // Can change pointer address, but can't change int
int const * const icpcpc = &icpcp;
"icpcpc is a const pointer to a pointer to a const pointer to a const int"
```

`nullptr`: Represents a `null`-Pointer. Is a *literal (prvalue)* and has type `nullptr_t`. Implicit conversion to any pointer type: "T *". Prefer `nullptr` over 0 and `NULL` *(no overload ambiguity, no implicit conversion)*.

Pointer vs. Reference

Pointers ...	*References ...*
can be `nullptr`	are always bound to an object
can be changed *(if not const)*	cannot be rebound to another object
require dereferencing with "*" or "->"	allow member access by "."
can be dangling *(pointing to a destroyed object)*	can be dangling *(referencing a destroyed object)*

Use raw pointers only to explicitly *model the possibility of a `nullptr`* *(requires a check like `auto foo(int* ptr) -> void { if (ptr) { ... } }`)* and for *modelling borrowing* only. Else, use *smart pointers*.

3.3. MEMORY ALLOCATION WITH NEW

```
new <type> <initializer>
```

Allocates memory for an instance of `<type>`. Returns a pointer to the object or array created on the heap of type `<type> *`. The arguments in the `<initializer>` are passed to the constructor of `<type>`. *Memory Leak if not removed with delete*. Avoid manual allocation, use RAII instead.

```
struct Point { Point(int x, int y) : x(x), y(y){} int x, y; };
auto createPoint(int x, int y) -> Point* { return new Point{x, y}; // constructor }
auto
auto createCorners(int x, int y) -> Point* { return new Point[2]{{0, 0}, {x, y}};
```

3.3.1. Placement new

CPPReference: new expression, *CPPReference: std::construct_at* `#include <memory>`

```
new (<location>) <type> <initializer>
```

Used for placing elements on the heap in the *location of a deleted element*. Does not allocate new memory *(Undefined Behavior!)* The memory of `<location>` needs to be suitable for construction of a new object and any element there *must be destroyed before*. Calls the constructor for creating the object at the given location and returns the memory location. Better use `std::construct_at()`.

```
auto ptr = new Point{9, 8};
// destruct Point here...
new (ptr) Point{7, 6};
delete ptr;

// Better:
std::construct_at(ptr, 7, 6);
```

3.4. MEMORY DEALLOCATION WITH DELETE

```
delete <pointer>
```

Deallocates the memory of a single object pointed to by the `<pointer-to-array>`. Calls the destructor of the destroyed type. `delete nullptr` does nothing. *Deleting the same object twice is Undefined Behavior!*

```
struct Point { Point(int x, int y) : x(x), y(y){} int x, y; };
auto foWithPoint(int x, int y) -> void {
  Point * pp = new Point{x,y}; // Member access here with "pp->"
  delete pp; // calls destructor and releases memory
}
```

3.4.1. Placement delete

Does not exist, but a destructor can be called explicitly, like any other member function.

```
$ * ptr = ...; ptr->~$();
```

Destroys the object, but does *not* free its memory. Called like any other member function. Better use `std::destroy_at(ptr)`. Useful for non-default constructible types.

3.4.2. Array Memory Deallocation with delete[]

```
delete[] <pointer-to-array>
```

Deallocates the memory of an array pointed to by the `<pointer-to-array>`. Calls the destructor of the destroyed objects inside the array. Also deletes *multidimensional arrays*. It is not necessary to know the exact amount of elements in the array. Undefined behavior on non-array types.
`Point <arr> = new Point[5][6]; delete[] arr;` */* do stuff */* `delete[] arr;`

3.5. NON-DEFAULT CONSTRUCTIBLE TYPES

A type is non-default-constructible when there is *no explicit or implicit default constructor*.
To create arrays of NDC types, allocate the plain memory and initialize it later *(Slides page 36 onward)*.

```
auto memory = std::make_unique<std::byte[]>(sizeof(Point) * 2 /* Array size */);
auto location = reinterpret_cast<Point>(memory.get());  // Address of first element
std::construct_at(location, 1, 2); // Equivalent to arr[0] = Point{1, 2} on regular types
auto value = elementAt(memory.get(), 0); // Access value via helper function
std::destroy_at(location);

auto elementAt(std::byte* memory, int index) -> Point& // helper function
  return reinterpret_cast<Point>>[memory][index]; // dereferences implicitly
}
```

Don't use an element if it is uninitialized. Destroy all elements before the memory is deallocated.
Use a `std::byte` array as memory for NDC Elements:
– **Static:** `std::array<std::byte, no_of_bytes> values_memory;` *(on stack, size known at compile-time)*
– **Dynamic:** `std::unique_ptr<std::byte[]> values_memory;` *(on heap, size known at run-time)*

3.6. CLASS-SPECIFIC OVERLOADING OF NEW/DELETE OPERATORS

Overloading `new` and `delete` for a class can inhibit heap allocation. This can be used to provide efficient allocation. Is useful with a memory pool for small instances or if thread-local pools are used. Can log or limit number of heap-allocated instances. *But in general, not advisable.*

```
struct not_on_heap { // Prevents heap allocation of this class
  static auto operator new(std::size_t sz) -> void * { throw std::bad_alloc(); }
  static auto operator new[](std::size_t sz) -> void * { throw std::bad_alloc(); }
  static auto operator delete(void *ptr) -> void noexcept { /* do nothing */ }
  static auto operator delete[](void *ptr) -> void noexcept { /* do nothing */ } };
```

3.7. READING DECLARATIONS

1. Declarations are read *starting by the declarator* (name)
2. *Read to the right* until a *closing parenthesis* is encountered
3. *Read to the left* until an *opening parenthesis* is encountered
4. *Jump out of the parentheses* and start over

Specifiers right to the declarator: Array Declarator *([])* and Function Parameter List *((<parameter declarations>))*
Specifiers left to the declarator: References *(&&, &)*, Pointers *(*)* and Types *(int)*
const: Applies to its left neighbor, if there is no left neighbor, it applies to its right neighbor:
`const int i; int const j;` Should always be written to the right of the type to avoid surprises *(i.e. with aliases)*.

```
void (* f)(int &, double)
f is a pointer to a function that takes a reference to int and a double, returning void.

int const * (* f[2][3]) [5];
f is an array of 2 elements of arrays of 3 elements of pointers to arrays of 5 elements of pointers to const int.

int (* f(int(*)(int)) [int]); // f?(int (*)(int) [int])
f is a function that takes a pointer to a function as argument *)* and returns a pointer to a function *[]*.
The function in the argument takes an int as argument *(a)* and returns an int *[a]*. The returned function takes an int as argument *(a)* and returns an int *[?]*. Always use type aliases on cases like this!
```

3.8. RESOURCE MANAGEMENT WITH RAII

CPPReference: RAII

Resource Acquisition Is Initialization is an alternative to allocating and deallocating a resource explicitly. Wraps allocation and deallocation in a class, uses regular constructor/destructor. Cleaned up at end of scope.

3.8.1. std::unique_ptr and std::make_unique

CPPReference: std::unique_ptr, *CPPReference: std::make_unique*

A unique pointer wraps a plain pointer, has zero runtime overhead. A custom deleter could be supplied if required. Always use `make_unique` for creation. Can create unbound arrays, but not fixed size arrays.
`std::unique_ptr<char> cPtr = std::make_unique<char>('*');`

3.8.2. Container Resource Handling

CPPReference: std::vector/T,Allocator::emplace, *CPPReference: std::stack/T,Container::emplace*

Constructs elements directly in a container, more efficient than moving them. Not available for `std::array`.
`std::stack<Point> vec{}; vec.emplace(3, 5); // std::vector requires position argument`

4. ITERATORS & TAGS

4.1. TAGS FOR DISPATCHING

CPPReference: std::... iterator_tag

If the *same operation* can be implemented more/less efficiently depending on the capabilities of the argument, *tags can be used to find the "best" implementation*. Tags are used to *mark capabilities* of associated types. They do not contain any members. In the example, different spaceships have different means of travel:

```
// provides travelThroughSpace(), the base functionality
struct SpaceDriveTag{};
template<typename> struct SpaceshipTraits { using Drive = SpaceDriveTag; };
// provides travelThroughSpace() and travelThroughHyperspace(), specialized template
struct SubspaceDriveTag : SpaceDriveTag{};
template<> struct SpaceshipTraits<GalaxyClassShip> { using Drive = SubspaceDriveTag; };

template <typename Spaceship>
auto travelToDispatched(Galaxy destination, Spaceship& ship, SpaceDriveTag) -> void {
  ship.travelThroughSpace(destination); // For Spaceships with SpaceDriveTag
}
template <typename Spaceship>
auto travelToDispatched(Galaxy destination, Spaceship& ship, SubspaceTag) -> void {
  ship.travelThroughHyperspace(destination); // For Spaceships with SubspaceTag
}
template <typename Spaceship> // The public API function, tags aren't publicly visible
auto travelTo(Galaxy destination, Spaceship& ship) -> void {
  typename SpaceShipTraits<Spaceship>::Drive drive{}; // get the Spaceship's drive tag
  travelToDispatched(destination, ship, drive); // call overloaded function
}
```

4.2. ITERATORS

CPPReference: Iterator library, *CPPReference: Iterator tags*

Different algorithms require different strengths of iterators. Iterators capabilities can be determined at compile time with *Iterator tag types (std::output_iterator_tag, std::input_iterator_tag etc.)*.
– **Outputiterator:** Write results *(to console, file etc.)*, without specifying an end *(used on std::ostream)*.
`operator *` returns an *lvalue reference* for assignment of the value.
– **Inputiterator:** Read sequence once *(used on std::istream)*.
`operator *` returns *const lvalue reference*, or *rvalue*.
– **Forwarditerator:** Read/write sequence, multi-pass *(used on std::forward_list linked-list)*.
`const operator *` returns *const lvalue reference* or rvalue, non-`const operator *` returns *lvalue*
– **Bidirectionaliterator:** Read/write, back and forth *(used on std::list double-linked-list)*.
– **Randomaccessiterator:** Read/write/indexed sequence *(used on std::vector)*.

You need to implement the members required by your `iterator_tag`.

```
struct IntIterator { // Provide these member types to align with STL iterators
  using iterator_category = std::input_iterator_tag; // iterator category
  using value_type = int; // Type of element the iterator iterates over
  using difference_type = ptrdiff_t; // specifies iterator distance
  using pointer = int *; // pointer type of the elements iterated over
  using reference = int &; // reference type of the elements iterated over
};
```

4.2.1. `iterator_traits<>`

CPPReference: std::iterator_traits

STL algorithms often want to *determine the type* of some specific thing related to an iterator. However, not all iterator types are actually classes. Default `iterator_traits` just pick the type aliases from those provided.

```
template <typename InputIterator, typename Tp>
auto count(InputIterator first, InputIterator last, const Tp& value)
  -> typename Iterator_traits<InputIterator>::difference_type { ... }
```

Specialization of `iterator_traits` also allows "naked pointers" to be used as iterators in algorithms.

```
template <typename _Tp> struct iterator_traits<Tp> { /* Provide regular iter usings */ }
```

4.2.2. Problems with the Stream Input Iterator

Reference Member and Default Constructor
When implementing a input iterator, we need to be able to create an `EOF` iterator. This dirty hack works, but the global variable to initialize the reference in an anonymous namespace is bad for multi-threading.

```
namespace { // global variable to initialize the reference in an anonymous namespace
  std::istringstream empty{};  // pseudo default
}
IntInputIter::IntInputIter() : input { empty } // Reference empty stream, also initialized
  // guarantee the empty stream is (good()
  input.clear(std::ios_base::eofbit); // mark empty stream as EOF
}

// Deref
// Deref
// Stream iter comparisons only make sense for testing if they can still be read from
auto IntInputIter::operator==(const IntInputIter & other) const -> bool {
  return !input.good() && !other.input.good(); }
```

4.2.3. Custom Iterator Example (Testat 2)

```
template<typename V>
struct Iterator_base {
  using value_type = BoundedBuffer::value_type;
  using reference = BoundedBuffer::reference;
  using const_reference = BoundedBuffer::const_reference;
  using size_type = BoundedBuffer::size_type;
  using difference_type = std::ptrdiff_t;
  using iterator_category = std::random_access_iterator_tag;

  auto operator++(Iterator_base const & other) const -> bool { ... }
  auto operator<=>(Iterator_base const & other) const -> std::strong_ordering { ... }

  auto operator+(int const) ->
```

6.4. USER-DEFINED LITERALS

CPPReference: User-defined literals

```
operator"" _UDLSuffix()"
```

Allows integer, floating-point, character, and string literals to produce objects of user-defined type by defining a user-defined suffix (*e.g. _kph*). The suffix must start with an underscore. It allows to add dimension, conversion, etc. If possible, define UDL operator functions as `constexpr`. Usually a *conversion function* like `safeToDouble()` and a *conversion operator* is needed. ***Rule:*** put overloaded UDL operators that belong together in a separate namespace.

```
namespace velocity::literals {
constexpr inline auto operator"" _kph(unsigned long long value) -> Speed<Kph>
  { return Speed<Kph>{safeToDouble(value)}; } // user defined literal operator, can be
                                              // called with Speed s = 80_kph;

template<typename T>
concept arithmetic = std::is_arithmetic_v<T>; // allows ints & floats in + operator

static constexpr inline auto safeToDouble(long double value) -> double {
  if (value > std::numeric_limits<double>::max()
    || value < std::numeric_limits<double>::lowest()) {
      throw std::invalid_argument{"Value must be within double range"};
  }
  return static_cast<double>(value);
}
} // namespace velocity::literals

struct Speed {
  constexpr explicit Speed(double value) : value{value} {};
  constexpr explicit operator double() const { return value; } // convert Speed -> double
  auto constexpr operator+(arithmetic auto rhs) -> decltype(rhs) { return value + rhs }
private:
  double value{};
}
auto constexpr operator+(arithmetic auto lhs, Speed rhs) -> decltype(lhs)
  { return rhs + lhs; } // Make + operator commutative (value + Speed calls Speed + value)
```

Signatures: `unsigned long long` for integral constants, `long double` for floating point constants, (`char const *`, `size_t len`) -> `std::string` for string literals, `char const *` for a raw UDL operator *(i.e. to convert into and from binary, is not constexpr).*

Standard Literal Suffixes: Do not use leading underscore: `std::string` *(s)*, `std::complex` *(i, if, df)*, `std::chrono::duration` *(ns, us, ms, s, min, h)*

6.4.1. Template UDL Operator

```
template <char...>
auto operator""_suffix() -> TYPE
```

Has an empty parameter list and a variadic template parameter. Characters of the literal are template arguments. Often used for interpreting individual characters. Since C++20, the template UDL operator works with string literals as well *(example and compile-time steps in slides on page 42 - 44).*

6.5. PREPROCESSOR

`hello.cpp` → preprocessor → `hello.i` → compiler → `hello.o` → linker → `hello` *(executable program)*

Object-like Macros

```
#define identifier replacement-list new-line
```

Identifier is a unique name, by convention in ALL_CAPS. Is valid until `#undef NAME`. *Replacement-list* is a possible empty sequence of preprocessor tokens. *New-line* terminates the replacement list.
Example: #define NUMBER_OF_ROWS 5

Function-like Macros

```
#define identifier ( identifier-list?, ?...? ) replacement-list new-line
```

Features an optional parameter list, containing only names. Params with a # prefix turn into string literals.

Includes: Textual inclusion of another file. *#include "path"* for including a header file from the same project or workspace, *#include <path>* for external includes.
Conditional Includes: Enable a section depending on a condition. *(example and macros in slides on page 52 - 62).*

```
#if constant-expression new-line          #ifndef identifier new-line
#elif constant-expression new-line        #ifndef identifier new-line
#else new-line
#endif new-line
```

7. MULTI-THREADING & MUTEX

`std::thread` to explicitly run a new thread, `std::async` to easily wrap a computation *(possibly with a result)*, `std::mutex` and co. to facilitate synchronization, *pthreads* is legacy. No portability guarantee.

7.1. API OF STD::THREAD

CPPReference: std::thread

```
auto main() -> int { std::thread greeter { []() /* lambda */ }; greeter.join(); }
```

A new thread is created and started automatically. Creates a new execution context. `join()` waits for the thread to finish, while `detach()` lets threads run in the background. Besides *lambdas, functions* or *functor objects* can also be executed in a thread. The *return value* of the function is *ignored*. Threads are *default-constructible* and *moveable*. *Caution:* Program terminates if thread gets destructed without calling `join()` or `detach()` before!

```
struct Functor {
  auto operator()() const -> void { std::cout << "Functor" << std::endl; }
};
auto function() -> void { std::cout << "Function" << std::endl; }
auto main() -> int {
  std::thread functionThread(function);
  std::thread functorThread(Functor{});
  functorThread.join(); functionThread.join(); }
```

Streams: Using global streams does not create data races, but sequencing of characters could be mixed.

`std::this_thread` helpers: *`get_id()` (An ID of the underlying OS thread), `yield()` (Allows OS to schedule another thread), `sleep_for(duration)/sleep_until(time_point)` (Suspends thread for/until time)*

7.1.1. Passing arguments to a `std::thread`

```
template<class Function, class... Args>
explicit thread(Function&& f, Args&&... args);
```

The *`std::thread` constructor* takes a function/functor/lambda and arguments to forward. You should pass all arguments by *value* to avoid data races and dangling references. *Capturing by reference* in lambdas creates shared data as well *(if you have to use them, don't declare them as mutable!)*

```
auto fibonacci(std::size_t n) -> std::size_t { /* ... */ }
auto printFib(std::size_t n) -> void { auto fib = fibonacci(n); /* print... */ }
auto main() -> int { std::thread function { printFib, 46 }; function.join(); }
```

Before the `std::thread` object is destroyed, you must *`join()` (wait until finished)* or *`detach()` (detach from the main thread and run in the background)* the thread, otherwise you get a runtime error.

7.2. STD::JTHREAD

CPPReference: std::jthread

RAII wrapper that automatically calls `join()`. Also supports external stop requests *(Similar to C#'s Cancellation Token. `t.request_stop()` sends the request, while `stop_requested()` checks if a stop request has been received).*

7.3. INTER-THREAD COMMUNICATION

CPPReference: std::mutex, CPPReference: std::shared_mutex, CPPReference: std::timed_mutex *CPPReference: std::recursive_mutex, CPPReference: std::recursive_timed_mutex*

Communication happens with *mutable shared state*. *Problem:* Data Race. *Solution:* Locking the shared access or make access atomic.

All *mutexes* provide the following operations:

- *Acquire*: `lock()`: blocking, `try_lock()`: non-blocking *(returns true if all locks could be acquired, false no locks acquired)*
- *Release*: `unlock()`: non-blocking

Two properties specify the capabilities. There is a mutex type for all four combinations *(see CPPR links)*:

- *Recursive*: Allow multiple nested acquire operations of the same thread *(prevents self-deadlock)*
- *Timed*: Also provide timed acquire operations *(`try_lock_for(duration)`, `try_lock_until(time)`)*

Reading operations don't need exclusive access. Only concurrent writes need exclusive locking. Use `std::shared_mutex/std::shared_timed_mutex` with `lock_shared()` to *read with multiple threads*. Shared mutex also have the exclusive lock methods from a regular mutex.

7.3.1. Acquiring / Releasing Mutexes

CPPReference: std::lock_guard *CPPReference: std::scoped_lock* *CPPReference: std::unique_lock* *#include <mutex>*
CPPReference: std::shared_lock *CPPReference: std::condition_variable* *#include <shared_mutex>*

Usually you use a lock that manages the mutex:

- *`std::lock_guard`*: RAII wrapper for a single mutex. Locks immediately when constructed, unlocks when destructed.
- *`std::scoped_lock`*: RAII wrapper for multiple mutexes. Locks immediately when constructed, unlocks when destructed. Acquires multiple locks in the constructor, avoids deadlocks by relying on internal sequence. Blocks until all locks could be acquired. *(`std::scoped_lock both(m, other.m);`)*
- *`std::unique_lock`*: Mutex wrapper that allows deferred and timed locking. Similar interface to timed mutex, allows explicit locking/unlocking. Unlocks when destructed.
- *`std::shared_lock`*: Wrapper for shared mutexes. Allows explicit locking/unlocking, unlocks when destructed.
- *`std::condition_variable`*: Similar to a Java Condition Variable. Waits for the condition and then notifies a potential change. `wait` will go into sleep until the notification has not been met yet. *(`wait(<mutex>, <predicate>)`, `wait_for()`, `wait_until()`, `notify_one()`, `notify_all()`)*

Standard Containers and Concurrency: There is no thread-safety wrapper for standard containers. Access to different individual elements from different threads is *not* a data race. Almost all other concurrent uses of containers are *dangerous*. `shared_ptr` copies to the same object can be used from different threads, but accessing the object itself can race if non-const *(reference counter is atomic).*

7.3.2. Thread-safe Guard Example (Testat 3)

Scoped Lock Pattern: Create a lock guard that (un)locks the mutex automatically. Every member function is mutually exclusive because of scoped locking pattern. *Strategized Lock Pattern:* Template Parameter for mutex type.

```
template <typename T, // Strategized Lock Pattern: User can change mutex type
          typename MUTEX = std::mutex, typename CONDITION = std::condition_variable>
struct threadsafe_queue {
  using guard = std::lock_guard<MUTEX>;
  using lock = std::unique_lock<MUTEX>;
  template <typename ForwardType>
  auto push(ForwardType &&t) -> void {
    guard lk{m}; a.push(std::forward<ForwardType>(t)); not_empty.notify_one();
  }
  auto pop() -> T {
    lock lk{m}; // wait requires timed locking therefore unique_lock
    not_empty.wait(lk, [this]{ return !q.empty(); }); // checked once, no busy wait
    T t = q.front();
    q.pop();
    return t;
  }
  auto try_pop(T & t) -> bool {
    guard lk{m}; if (q.empty()) { return false; } t = q.front(); q.pop(); return true;
  }
  // call container empty, not this-empty, would cause deadlock
  auto empty() const -> bool { guard lk{m}; return q.empty(); }
private:
  std::queue<T> q{};
  mutable MUTEX m{}; // mutable to unlock in const member functions
  CONDITION not_empty{};
}; // Mutex & condition_variable don't need to be swap()-ed. But notify them of changes!
```

7.4. RETURNING RESULTS FROM THREADS

We can use shared state to "return" results. *Acquire lock* in producer, *write* the shared result, *wait* for the result, *read* the result. We cannot communicate exceptions!

```
auto main() -> int {
  auto mutex = std::mutex{}; auto finished = std::condition_variable{}; auto shared = 0;
  auto thread = std::thread{[&]{
    std::this_thread::sleep_for(2s);
    auto guard = std::lock_guard{mutex}; // Lock the mutex
    shared = 42; finished.notify_all(); // Wakes up all threads waiting on condition
  }}; // Mutex is unlocked when guard goes out of scope
  std::this_thread::sleep_for(1s);
  auto lock = std::unique_lock{mutex}; // Lock the mutex
  finished.wait(lock); // Release mutex, wait until thread unlocks mutex, lock it again
  std::cout << "The answer is: " << shared << "\n"; thread.join(); }
```

7.4.1. std::future

CPPReference: std::future *#include <future>*

Future represents results that may be computed asynchronously. They allow us to *wait until the result is available…*

- `wait()`: blocks until available,
- `wait_for(<timeout>)`: Blocks until available or timeout elapsed,
- `wait_until(<time>)`: blocks until available or the timepoint has been reached

…and then *get the result*

- `get()`: blocks until available and returns the result value or throws

Their destructor may wait for the result to become available, unless they have no shared state.

7.4.2. std::promise

CPPReference: std::promise *#include <future>*

Promises are one origin of futures. They allow us to *obtain* a future using `get_future()` and *publish* results or errors *(`set_result(<result_value>)` – set the associated future's result to result_value, `set_exception(<exception_pointer>)` – set the associated future's exception)* .

```
auto main() -> int {
  using namespace std::chrono_literals;
  std::promise<int> promise{}; auto result = promise.get_future();
  auto thread = std::thread{
    [&]{std::this_thread::sleep_for(2s); promise.set_value(42); }
  }
  std::this_thread::sleep_for(1s);
  std::cout << "The answer is: " << result.get() << "\n"; thread.join(); }
```

7.5. STD::ASYNC

CPPReference: std::async *#include <future>*

Ready made *solution for computing asynchronously*. It allows us to *return our result* from the computation function. Additionally, it *catches all exceptions* and propagates them.

```
template<typename Function, typename ...Args>
auto async(Function&& f, Args&&... args) -> std::future</* implicitly from f */>;

auto main() -> int {
  auto the_answer = std::async([] { /* calculate a while ... */ return 42; });
  std::cout << "The answer is: " << the_answer.get() << "\n"; }
```

This function *returns a std::future* that will store the result. `get()` waits for the result to be available. `std::async` can take an argument of type *`std::launch` (launch policy): `std::launch::async`* launches a new thread and executes it regardless of if we need the result or not, *`std::launch::deferred`* defers execution until the result is obtained from the `std::future` *(lazy evaluation)*, which computes the result on the thread calling `get()`. The default policy is `std::launch::async | std::launch::deferred` *(environment dependent).*

8. MEMORY MODEL & ATOMICS

The *C++ Standard* defines an *abstract machine* which describes *how a program is executed*. *Platform specifics are no longer relevant* with this abstraction. Represents the *"minimal visible computer"* required to execute a C++ program. The abstract machine defines in *what order initialization* takes place and in *what order* a program is executed. It defines what a *thread* is, what a *memory location* is, how threads *interact* and what constitutes a *data race*.

Memory Location: An object of scalar type *(Arithmetic, pointer, enum, std::nullptr_t. These values have the same amount of bits on the architecture defined, like int-64)*.
Conflict: Two expression evaluations run in parallel. Both access the same Memory Location *(at least one write, one read)*
Data Race: The program contains two conflicting actions. Undefined Behavior!

8.1. MEMORY MODEL

The *C++ Memory Model* defines when the effect of an operation is visible to other threads and how and when operations might be reordered. The memory ordering define when effects become visible.

- *Sequentially-consistent*: Same as code ordering and the default behavior
- *Acquire/Release*: Weaker guarantees than sequentially-consistent
- *Consume*: Discouraged, slightly weaker than acquire-release
- *Relaxed*: No guarantees besides atomicity

Visibility of effects: *(If one thread modifies a variable, under what conditions is another thread guaranteed to see that modification?)*
sequenced-before: (a comes before B within a single thread), synchronizes-with: (inter-thread sync, event that makes another thread see it, e.g. with Mutex), happens-before: (A happens-before B if A is sequenced-before or synchronizes-with B),

Read/Writes in a single statement are "unsequenced": `std::cout << ++i << ++i; // output unknown`

8.1.1. Atomics

CPPReference: std::atomic *CPPReference: std::atomic_flag* *CPPReference: std::memory_order* *#include <atomic>*

Template class to create atomic types. Atomics are guaranteed to be *data-race free*. There are several specializations in the standard library. The most basic type *`std::atomic_flag`* is guaranteed to be lock-free.

```
auto outputWhenReady(std::atomic_flag & flag, std::ostream & out) -> void {
  // start critical section
  while (flag.test_and_set()) /* set flag to true and returns old value */ { yield(); }
  out << "Here is thread: " << get_id() << std::endl;
  flag.clear(); // sets the flag to false
  // and critical section
}
auto main() -> int {
  std::atomic_flag flag { };
  std::thread t { [&flag] { outputWhenReady(flag, std::cout); } };
  outputWhenReady(flag, std::cout); t.join(); }
```

When creating your *own atomic type* with `std::atomic<T>`, the atomic member operations are:

- *`void store(T)`*: Set the new value
- *`T load()`*: Get the current value
- *`T exchange(T)`*: Set the new value and return the old one
- *`bool compare_exchange_weak(T & expected, T desired)`*: Compare `expected` with current value, if equal replace the current value with `desired`, otherwise replace `expected` with current value. May spuriously fail
- *`bool compare_exchange_strong(T & expected, T desired)`*: Cannot fail spuriously, but slower

Applying Memory Orders: All atomic operations take an additional argument to specify the memory order *(Type `std::memory_order`)*. **Example:** `flag.clear(std::memory_order::seq_cst);` *(clear may not be reordered)*

- **Sequential Consistency (`seq_cst`):** Global execution order of operations. Every thread observes the same order. This is the default behavior. The latest modification will be available to a read operation.
- **Acquire (`acquire`):** No reads or writes in the current thread can be reordered *before* this load. All writes in other threads that *release the same atomic* are visible in the current thread. *Not guaranteed to see latest write* (Half Fence), but ordering is consistent.
- **Release (`release`):** No reads or writes in the current thread can be reordered *after* this store. All writes in the current thread are visible in other threads that *acquire the same atomic*.
- **Acquire/Release (`acq_rel`):** Guaranteed to work on the latest value unlike Acquire, used for Read-Modify-Write operations (Full Fence) e.g. `test_and_set()`.
- **Relaxed (`relaxed`):** Does not give promises about sequencing. No data-races for atomic variables. Order can be inconsistent *(parallel `load()`/`store()`)* (Lost Updates), but may be more efficient. Difficult to get right.
- **Release/Consume:** Do not use! Data-dependency, hard to use. Better use acquire.

8.1.2. Custom Types with `std::atomic`

Custom types need to be trivially copyable. You *cannot have* a custom copy ctor, move ctor, copy assignment or move assignment. Object can only be accessed as a whole. No member access operator.

8.2. VOLATILE

```
volatile int num{0};
```

Volatile in C++ is *different* from volatile in Java and C#. Load and store operations of volatile variables must *not be elided*, even if the compiler cannot see any visible side-effects within the same thread. Prevents the compiler from *reordering* within the same thread *(but the hardware might reorder instructions anyway)*. Useful when accessing *memory-mapped hardware*. *Never use for inter-thread communication!*

8.3. INTERRUPTS

Interrupts are *events originating from underlying system* which *interrupt the normal execution* flow of the program. Depending on the platform, they *can be suppressed*. When an interrupt occurs, a *previously registered function* is called *("Interrupt Service Routines" – ISRs)*. Should be *short* and must *run to completion*. After the interrupt was handled, execution of the program resumes.

Data *shared* between an ISR and the normal program execution needs to be *protected*. All accesses must be *atomic*, modifications need to become *visible*. `volatile` helps because it suppresses compiler optimizations. Interrupts may need to be *disabled temporarily* to guarantee atomicity.

9. NETWORK & ASYNC

Sockets are an abstraction of endpoints for communication.

- **TCP Sockets** are reliable, stream-oriented and require a connection setup
- **UDP Sockets** are unreliable, datagram-oriented and do not require a connection setup.

Connection-Oriented Communication Pattern using Sockets:

Socket: Create a new communication end point
Bind: Attach a local address to a socket *(for Peer)*
Listen: Announce willingness to accept connections
Accept: Block caller until a connection request arrives
Connect: Actively attempt to establish a connection
Send: Send some data over the connection
Receive: Receive some data over the connection
Close: Release the connection

9.1. DATA SOURCES / BUFFERS

Transmit/Receive functions need *source/destination* buffers. The ASIO library doesn't manage your memory!
Fixed Size Buffers: `asio::buffer()`. Must provide at least as much memory as will be read. Can use several standard containers as a backend. Pointer + Size combinations are available.
Dynamically Sized Buffers: `asio::dynamic_buffer()` Use if you do not know the required space. Can work with `std::string` and `std::vector`.
Streambuf Buffers: `asio::streambuf`. Works with `std::istream` and `std::ostream`

9.2. ASIO LIBRARY

CPPReference: Extensions for networking *#include <asio.hpp> (if installed)*

9.2.1. Example: Synchronous TCP Client Connection with ASIO

Socket: All ASIO Operations require an I/O context. Create a TCP Socket using the context. There are asynchronous and synchronous functions to communicate with sockets.

```
asio::io_context context{}; // multiple contexts possible
asio::ip::tcp::socket socket{context}; // multiple sockets per context possible
```

Connect: If the IP address is known, an endpoint can be constructed easily. `socket.connect()` tries to establish a connection to the given endpoint.

```
auto address = asio::ip::make_address("127.0.0.1");
auto endpoint = asio::ip::tcp::endpoint{address, 80}; socket.connect(endpoint);
```

A resolver resolves the host names to endpoints. `asio::connect()` tries to establish a connection.

```
asio::ip::tcp::resolver resolver{context};
auto endpoints = resolver.resolve(domain, "80"); asio::connect(socket, endpoints);
```

Write: `asio::write()` sends data to the peer the socket is connected to. It returns when all data is sent or an error occurred *(`asio::async_write`)*.

```
std::ostringstream request{};
request << "GET / HTTP/1.1\r\n";
request << "Host: " << domain << "\r\n"; request << "\r\n";
asio::write(socket, asio::buffer(request.str())); // Blocks until data is sent
```

Read: `asio::read()` receives data sent by the peer the socket is connected to. It returns when the read-buffer is full, when an error occurred or when the stream is closed. The error code is set if a problem occurs, or the stream has been closed *(`asio::error::eof`)*.

```
constexpr size_t bufferSize = 1024; std::array<char, bufferSize> reply{};
asio::error_code errorCode{};
auto readLength = asio::read(socket, asio::buffer(reply.data(), bufferSize), errorCode);
```

Advanced Reading: `asio::read` also allows to specify completion conditions.

- *`asio::transfer_all()` (Default behavior, transfer all data or until buffer is full)*
- *`asio::transfer_at_least(std::size_t bytes)` (Read at least bytes number of bytes, may transfer more)*
- *`asio::transfer_exactly(std::size_t bytes)` (Read exactly bytes number of bytes)*

`asio::read_until` allows to specify conditions on the data being read.

- Simple matching of characters or strings *(read until '\n')* or more complex matching using `std::regex`
- Allows to specify a callable object *(prototype → can be iterated over, returns seen data if predicate true. Expects std::pair<iterator, bool> operator()(iterator begin, iterator end))*

Close: `shutdown()` closes the read/write stream associated with the socket. The destructor cancels all pending operations and destroys the object.

```
socket.shutdown(asio::ip::tcp::socket::shutdown_both); // close read and write end
socket.close();
```

9.2.2. Example: Synchronous TCP Server with ASIO

Socket, Bind & Listen: An *acceptor* is a special socket responsible for establishing incoming connections. Is bound to a given local end point and starts listening automatically.

```
asio::io_context context{};
asio::ip::tcp::endpoint localEndpoint{asio::ip::tcp::v4(), port};//uses any available IPv4
asio::ip::tcp::acceptor acceptor{context, localEndpoint};
```

Accept: `accept()` blocks until a client tries to establish a connection *(with connect)*. It returns a new socket through which the connected client can be reached.

```
asio::ip::tcp::endpoint peerEndpoint{}; // information about client
asio::ip::tcp::socket peerSocket = acceptor.accept(peerEndpoint);
```

9.2.3. Async communication

Handling multiple requests simultaneously: Using synchronous operations blocks the current thread. *Asynchronous Operations* allow further processing of other requests while the async operation is executed. Most OS support asynchronous IO operations.

1. The program *invokes an async operation* on an OS object *(socket)* & *passes a completion handler* callback.
2. The I/O object *delegates* the operation and the callback to its *`io_context`*.
3. The OS *performs* the asynchronous operation.
4. The OS *signals* the `io_context` that the operation has been *completed*.
5. When the program *`io_context::run()` runs* the *remaining* asynchronous operations are *performed* *(wait for the result of the operating system)*.
6. Still inside the `io_context::run()` the *completion handler* is called to handle the *result* of the asynchronous operation.

9.2.4. Asynchronous Read/Write on Sockets

- *Async read operations:* `asio::async_read, asio::async_read_until, asio::read_at` *(certain position in buffer)*
- *Async write operations:* `asio::async_write, asio::async_write_at`

They return immediately. The operation is processed by the executor associated with the stream's `asio::io_context`. A completion handler is called when the operation is done.

```
// Reads from async stream into buffer until '\n' is reached, then calls completion handl.
auto readCompletionHandler = [] (asio::error_code ec, std::size_t len_of_read) { /*...*/ }
asio::async_read_until(socket, buffer, '\n', readCompletionHandler);

// Writes data from a buffer to async stream until data has been written
// an error occurs. Then calls the completion handler.
auto writeCompletionHandler = [] (asio::error_code ec, std::size_t len_of_write) {/*...*/}
asio::async_write(socket, buffer, writeCompletionHandler);
```

9.2.5. Asynchronous Acceptor

Create an *accept handler* that is called when an incoming connection has been established. The second parameter is the socket of the newly connected client. A session object is created on the heap to handle all communication with the client. `accept()` is called to *continue new inbound connection attempts*. The accept handler is registered to handle the next accept asynchronously.

The constructor creates the server. It initializes its acceptor with the given `io_context` and port. It calls `accept()` for registering the accept handler and does not block. To use it, create an `io_context` and the `asio::ip::tcp::acceptor` server. The executor will run until no async operation is left thanks to `async_accept()`. It is important that *the server lives as long as async operations* on it are processed.

```
struct Server { using tcp = asio::ip::tcp;
  Server(asio::io_context &c, unsigned short port)
    : acceptor{c, tcp::endpoint{tcp::v4(), port}} { accept(); }
private:
  auto accept() -> void {
    auto acceptHandler = [this](asio::error_code ec, tcp::socket peer) {
      if (!ec) { // start new session if no error has occurred
        auto session = std::make_shared<Session>(std::move(peer));
        session->start();
      }
      accept(); // call accept again to continue accepting new connections.
    };          // If not done only one client session request will ever be handled
    acceptor.async_accept(acceptHandler); // Register acceptHandler to handle next accept
  }
  tcp::acceptor acceptor;
};
auto main() -> int {
  asio::io_context context{}; Server server{context, 1234}; context.run();
}
```

9.2.6. Session with Asynchronous IO

The constructor stores the socket with the client connection. *`start()`* initializes the first async read, *`read()`* invokes reading. *`write()`* invokes async writing. The fields store the data of the session. *`enable_shared_from_this`* is needed because the session object would die at the end of the accept handler, therefore it needs to be allocated on the heap. The *`handlers need to keep the object alive`* by pointing on it on the heap through their captured `self` pointer. If there is no pointer to the object left, it gets destroyed.

```
struct Session : std::enable_shared_from_this<Session> {
  explicit Session(asio::ip::tcp::socket socket); auto start() -> void { read(); }
private:
  auto read() -> void; auto write(std::string data) -> void;
  asio::streambuf buffer{}; std::istream input{&buffer}; asio::ip::tcp::socket socket;
};
// Code in Accept handler:
auto session = std::make_shared<Session>(std::move(peer)); session->start();
accept(); // if not called, only single connection possible
// Code in Read handler:
auto Session::read() -> void { auto readCompleteHandler = [self = shared_from_this()] /*...*/}
// Code in Write handler:
auto Session::write(std::string input) -> void {
  auto data = std::make_shared<std::string>(input);
  auto writeCompletionHandler = [self = shared_from_this(), data] /*...*/}
```

9.2.7. Async Operation without Callbacks

Async operations can work "without" callbacks. Specify "special" objects as callbacks.

- *`asio::use_future`*: Returns a `std::future<T>`. Errors are communicated via exception in future
- *`asio::detached`*: Ignores the result of the operation
- *`asio::use_awaitable`*: Returns a `std::awaitable<T>` that can be awaited in a coroutine. Complicated!

9.3. SIGNAL HANDLING

CPPReference: Standard library header <csignal> *#include <csignal>*

Most OS support signals. Signals provide asynchronous notifications. They are used to gracefully terminate a program, communicate errors, notify about traps *("it's a trap!")*

`SIGTERM` (Termination requested), `SIGSEGV` (Invalid memory access), `SIGINT` (User interrupt), `SIGILL` (Illegal instructions), `SIGABRT` (Abnormal termination), `SIGFPE` (Floating-point exception)

9.3.1. Signal Handling in ASIO

`asio::signal_set` defines a set of signals to wait for. Handlers can be set up with `signal_set.wait()`/`signal_set.async_wait()` that take a lambda. The signal handler receives the signal that occurred and an error if the wait was aborted: *([&](auto error, auto sig){...}. Useful to cleanly stop server applications.*

9.4. ACCESSING SHARED DATA

Multiple async operations can be in flight. All completion handlers are dispatched through `asio::io_context` and run on a thread executing `io_context.run()`. Multiple threads can call `run()` on the same `asio::io_context`. This results in a possible data race.

9.4.1. Strands

Strands are a mechanism to *ensure sequential execution* of handlers.

- *Implicit strands:* If only one thread calls `io_context.run()` or program logic ensures only one operation is in progress at a time.
- *Explicit strands:* For multiple threads on the same `io_context`. Objects of type `asio::strand<...>`. Created using `asio::make_strand(executor)` or `asio::make_strand(execution_context)`. Applied to handlers using `asio::bind_executor(strand, handler)`.

```
// globally accessible
auto results = std::vector<int> {}; auto strand = asio::make_strand(context);
// in connection class
asio::async_read(socket, asio::buffer(buffer),
  asio::bind_executor(strand, [&](auto err, auto bytes) { // wrap access in bind_executor
    auto result = parse(buffer); results.push_back(result); /* by bye bye data race */ }));
```

10. ADVANCED LIBRARY DESIGN

10.1. EXCEPTION SAFETY

There is code that *handles* exceptions, code that *throws* exception, and *exception neutral code (does not throw, does not catch, just propagate exceptions)*. Exception safety is important in generic code that manages resources or data structures *(might call user-defined operations, must not garble its data structures and must not leak resources)*

The *deterministic lifetime model of C++* requires exception safety. When an exception is thrown, "stack unwinding" ends the lifetime of temporary and local objects. Throwing an exception while another exception is "in flight" in the same thread causes the program to terminate.

10.1.1. Safety Levels (from highest to lowest)

- *noexcept aka no-throw:* Will never throw an exception *(and is always successful)*. Very hard to achieve, sometimes even impossible, e.g. when dealing with memory allocation. *(Examples: Swap, Move Constructor, Move Assignment Operator, `std::vector<T>::clear()`)*
- *Strong exception safety:* Operation succeeds and doesn't throw, or nothing happens but an exception is thrown *(transaction, compare DB transactions)*. Hard to achieve. *(Examples: Copy Constructor, Copy Assignment Operator implementing the Copy-Swap Idiom)*
  - `auto push(value_type const & elem) -> void {`
    `  value_type val{elem}; // might throw here due to copy operation failing`
    `  tail_ = (tail_ + 1) % (capacity() + 1); elements_++; /*elements not yet changed*/}`
- *Basic exception safety:* Operations can throw an exception but data structures and resources in use are in a valid state *(guarantees invariance)*, but operations might be only half-done *(i.e. if copy throws, but internal members have not been adjusted yet)*
- *No guarantee:* Often unintentional, but happens. Invalid or corrupted data when an exception is thrown.

A function can only be as exception-safe as the weakest sub-function it calls!

	Invariant OK	*All or Nothing*	*Will Not Throw*
No Guarantee	×	×	×
Basic Guarantee	✓	×	×
Strong Guarantee	✓	✓	×
No-Throw Guarantee	✓	✓	✓

10.1.2. noexcept Keyword

CPPReference: noexcept/noexcept(bool)

Tells the compiler that this function will never throw. `noexcept` belongs to the function signature. You *cannot overload* on `noexcept`. *`noexcept(expression)`* can be used to determine the *"noexceptiness"* of an expression, without computing it. `noexcept(noexcept(f()))` means "Outer function is `noexcept`, if function `f()` is also `noexcept`".

The compiler might *optimize* a call of a `noexcept` function *better*. But if you throw an exception from a `noexcept` function, `std::terminate()` will be called.

10.1.3. Member Functions that should not throw

- *Destructors* must not throw when used during stack unwinding
- *Move construction* and *move assignment* better not throw *(This is why it often uses swap internally)*
- *swap* should not throw *(`std::` swap requires non-throwing move constructor and move assignment)*
- *Copying* might throw, which may needs to be allocated

10.1.4. Standard Library Helpers

CPPReference: std::move, std::forward *#include <utility>*

It may be hard for a container to implement its moves operations if the element type does not support `noexcept`-move. Use `std::move_if_noexcept` instead: If `noexcept`, then move.

```
explicit Box(T && t) noexcept(noexcept(T(std::move_if_noexcept(t)))
  : value{std::move_if_noexcept(t)} { ... }
```

There are other helpers like `is_nothrow_...`: `constructible`, `move_constructible`, `default_constructible`, `assignable`, `move_assignable`, `copy_assignable`, `destructible`, `copy_constructible`, `swappable`

10.1.5. Wide and narrow contracts

- *Wide Contract:* A function that can handle all argument values of the given parameter types successfully. It cannot fail and should be specified as `noexcept(true)`. `this`, globals and external (heap) resources are also possible parameters.
- *Narrow Contract:* A function that has preconditions on its parameters, e.g. *int* parameter must not be negative. Even if not checked and no exception is thrown, those functions *should not be noexcept*. This allows later checking and throwing.

10.2. PIMPL IDIOM (POINTER TO IMPLEMENTATION IDIOM)

CPPReference: Pimpl

Opaque/Incomplete Types: Name known *(declared)*, but not the content. Introduced by a forward declaration. Can be used for pointers and references, but it cannot dereference values, access members or allocate memory without a later definition.

```
struct S; // Forward Declaration
int foo(S & s) -> void { Foo(s); /* S s{}; is invalid */}
struct S{}; // Definition
auto main() -> int { S s{}; Foo(s); /* s can now be used */ }
```

Problem: Internal changes in a class' definition require clients to re-compile *(e.g. changing a type of a private member variable)*. **Solution:** Create a "Compilation Firewall". Allow changes to implementation without the need to recompile users. It can be used to shield client code from implementation changes, meaning you must not change header files your client relies upon.

Put in the "exported" header file a class consisting of a "Pointer to Implementation" plus all public members. The private members reside only within the pimpl class. Implement the member of the pimpl class as normal.

With *`std::shared_ptr<class Impl>`* we can use a minimal header and hide all details in the implementation `WizardImpl`. The `WizardImpl` class called from the header implements all calls to `WizardImpl`.

```
// wizard.hpp: Minimal Header                    // WizardImpl.cpp: Full implementation of
class Wizard {                                   // WizardImpl (all private members, not shown
  std::shared_ptr<class WizardImpl> pImpl;       // here) and Wizard (PImpl & public methods)
public:                                          Wizard::Wizard(std::string name)
  Wizard(std::string name = "Rincewind");          pImpl{std::make_shared<WizardImpl>(name)}{}
  auto doMagic(std::string wish)
    -> std::string;                              auto Wizard::doMagic(std::string wish)
};                                                 -> std::string {
                                                   return pImpl->doMagic(wish);
                                                 }
```

With *`std::unique_ptr<class Impl>`* we need to define the destructor of `Wizard` after the definition of `WizardImpl`. The compiler can't move the destructor by himself.

```
// wizard.hpp                                    // WizardImpl.cpp
class Wizard {                                   class WizardImpl {/* ... */ };
  std::unique_ptr<class WizardImpl> pImpl;       //
public:                                          Wizard::~Wizard() = default; // default dtor
  Wizard(std::string name);
  ~Wizard();// define explicitly
  auto doMagic(std::string wish)
    -> std::string; };
```

Because the default deleter of `std::unique_ptr` can't delete an incomplete type, we need to define the destructor explicitly.

10.2.1. Design Decisions with Pimpl Idiom

How should objects be copied?

- *No copying - only moving:* `std::unique_ptr<class Impl>` *(declare destructor and move operations & default)*
- *Shallow copying:* `std::shared_ptr<class Impl>` *(sharing the implementation)*
- *Deep copying:* `std::unique_ptr<class Impl>` *(with DIY copy constructor of Impl, default for C++)*

Never do `pImpl == nullptr`, and do not inherit from Pimpl class.

11. HOURGLASS INTERFACES

11.1. APPLICATION BINARY INTERFACES (ABI)

ABIs define *how programs interact* on a *binary level (Names of structures and functions, calling conventions, instruction sets)*. *C++ does not define* any specific ABI, because they are *tightly coupled* to the platform. They change between OSes, compiler versions, library versions, etc. Different STL implementations are *(usually)* incompatible.

Use *C[89]* as an "intermediate" layer, as "C frontend for our C++ code". This is a *extremely stable ABI*. No namespaces, no name mangling. But there are also no member functions, no exceptions and no templates.

11.1.1. Example: Wizard Class

```
// C++ Library
struct Wizard {
  Wizard(std::string name = "Rincewind") : name{name}, wand{} {}
  auto doMagic(std::string const & wish) -> char const *;
  auto learnSpell(std::string const & newSpell) -> void;
  auto mixAndStore(std::string const & potion) -> void;
  auto getName() const -> char const * { return name.c_str(); }
};
// C++ Client / Tests: Work normally as you would expect
using wizard_client::Wizard;
TEST(canCreateDefaultWizard) {
  Wizard const magician{};
  ASSERT_EQUAL("Rincewind", magician.getName());
}
```

Background C API: Abstract data types can be *represented by pointers*. Ultimate abstract pointer: `void *`. Member functions map to functions taking the abstract data type pointer as first argument. Requires *factory and disposal functions* to manage object lifetime. Strings can only be represented by `char *`. Make sure to not return pointers to temporary objects. Exceptions do not work across a C API, use a `Error` struct.

```
// C Header / C API DLL (Wizard.h)
#ifdef __cplusplus // "extern" only compiles with a CPP, not C compiler
extern "C" { // mark header file as C to disable name mangling
#endif
typedef struct Wizard * wizard; // Wizard can only be accessed through pointers
typedef struct Wizard const * cwizard;
typedef
```